

METIS-X: A Cache-Conscious Symbol Table Architecture for Resource-Constrained Embedded Toolchains

Adhyan Jain
SCOPE, VIT Vellore
Vellore, India

Shubhi Singh
SCOPE, VIT Vellore
Vellore, India

Bhuvaneswari M
SCOPE, VIT Vellore
Vellore, India

Abstract—Symbol tables are a core compiler data structure whose physical memory footprint and lookup latency directly govern execution efficiency under severe embedded resource constraints. Modern 64-bit systems rely on `std::unordered_map` with Short String Optimization (SSO), storing small identifiers (≤ 15 bytes) without secondary heap allocations. In adaptive representation studies (Phase I, `SymTabV3`), front-coded block compression achieved a 20.2% physical heap reduction on large workloads but incurred a $1.824\times$ p_{95} tail-latency regression due to per-lookup reconstruction costs.

To overcome this fundamental boundary, we present **METIS-X** (Phase II), a cache-conscious, compiler-aware flat symbol-table architecture. **METIS-X** eliminates dictionary reconstruction from the hot lookup path by storing short identifiers (≤ 12 bytes) directly inside a fixed 32-byte slot, backed by Robin Hood open-addressing displacement and LIFO scope-lifetime frame slot recycling. We evaluate **METIS-X** using physical allocator heap tracking (`malloc_usable_size`) under $R = 7$ independent repetitions with CPU core pinning (`taskset -c 0`). On the primary ZEPHYR RTOS workload (2.6M events, 228,739 unique symbols), **METIS-X** achieves a 40.2% physical final heap reduction (24.95 MB vs 41.74 MB) and a 45.7% p_{95} lookup latency reduction ($0.082\mu\text{s}$ vs $0.151\mu\text{s}$) relative to the embedded conventional baseline. On ESP-IDF, **METIS-X** achieves a 17.6% heap reduction (35.15 MB vs 42.65 MB) and a 56.7% p_{95} latency reduction ($0.081\mu\text{s}$ vs $0.187\mu\text{s}$). Instrumentation across 2.8M lookups proves zero secondary heap allocations on the hot path.

Index Terms—Symbol Table, Compiler Memory Management, Embedded Systems, Cache-Conscious Data Structures, Open Addressing, Robin Hood Hashing, Short String Optimization.

I. INTRODUCTION

Compilers and language interpreters repeatedly create, resolve, and reclaim symbolic bindings during lexical, syntactic, and semantic analysis [1]. A symbol table maps identifier names to binding metadata across nested lexical scopes. In resource-constrained embedded systems—such as microcontroller toolchains, real-time operating systems (RTOS), and on-device interpreters—available RAM is severely limited.

In desktop and server host compilers (e.g., LLVM [2]), standard C++ string implementations rely on Short String Optimization (SSO) [3], [4]. Identifiers up to 15 bytes are stored directly inside the string control block, avoiding heap allocations. Consequently, conventional hash maps (`std::unordered_map<std::string,`

`SymbolMeta>`) pay zero secondary heap string allocation cost for short names.

In Phase I of this research (`SymTabV3`), we evaluated adaptive 3-tier representation routing (INLINE, INTERNED, front-coded COMPRESSED). While `SymTabV3` achieved a 20.2% physical RAM reduction on ZEPHYR RTOS, it failed the $1.25\times$ tail-latency gate ($p_{95} = 0.228\mu\text{s}$ vs $0.125\mu\text{s}$, a $1.824\times$ regression) due to unavoidable string reconstruction overhead during lookup.

This negative result yields the central research question of Phase II:

Can a cache-conscious flat open-addressing symbol table eliminate dictionary reconstruction overhead to achieve simultaneous physical heap and p_{95} latency reductions over embedded conventional baselines?

We answer this in the affirmative by presenting **METIS-X**. We make the following main contributions:

- 1) **Cache-Conscious Architecture (METIS-X)**: We present a 32-byte flat slot design with ≤ 12 B inline name storage, Robin Hood displacement, 32-bit hash cache guards, and LIFO scope-lifetime frame slot recycling.
- 2) **Zero-Allocation Hot Path**: We instrument and prove zero secondary heap allocations across 2.8M lookup calls in real embedded compiler traces.
- 3) **Joint Memory/Latency Wins**: On ZEPHYR RTOS, **METIS-X** reduces physical final heap by 40.2% (24.95 MB vs 41.74 MB) and p_{95} lookup latency by 45.7% ($0.082\mu\text{s}$ vs $0.151\mu\text{s}$). On ESP-IDF, **METIS-X** reduces heap by 17.6% (35.15 MB vs 42.65 MB) and p_{95} latency by 56.7% ($0.081\mu\text{s}$ vs $0.187\mu\text{s}$).
- 4) **Resolution of Phase-I Latency Failure**: On ZEPHYR, **METIS-X** is $2.68\times$ faster at p_{95} than `SymTabV3` ($0.082\mu\text{s}$ vs $0.220\mu\text{s}$) while consuming 3.27 MB less heap (24.95 MB vs 28.22 MB).
- 5) **Component Ablation & Failure Limits**: We perform a 5-level component ablation ($A_0 \rightarrow A_5$) proving that inline slot storage with scope recycling cuts physical heap by 89% over flat open addressing with per-entry heap strings, and quantify performance limits on long-

identifier fallback paths.

II. BACKGROUND: PHASE I LESSONS

A. Phase I Adaptive Compression (SymTabV3)

In Phase I, SymTabV3 organized entries into a scope-managed arena with 3-tier routing: `INLINE` (≤ 12 B), `INTERNED` (global pool index), and `COMPRESSED` (front-coded blocks). While front coding effectively reduced character redundancy, every lookup to a compressed symbol required backward anchor scanning and byte decoding:

$$T_{\text{lookup}} = T_{\text{hash}} + T_{\text{probe}} + T_{\text{reconstruct}} + T_{\text{compare}} \quad (1)$$

The $T_{\text{reconstruct}}$ term dominated tail latency, causing the $1.824 \times p_{95}$ regression on ZEPHYR.

B. Phase I Disclosed Negative Result (SymTabV4)

To test whether moving scalar metadata (`accessCount`, `lastAccessEpoch`) out of `PackedEntry` (shrinking it from 32 B to 24 B) into a sparse side table (`HotMetaTable`) improved byte density, we built SymTabV4. Across 20 host corpora, SymTabV4 lost to Conventional on 20/20 workloads because side-table allocation floors (T_{table}) outweighed scalar field savings across realistic representation mixes.

III. METIS-X ARCHITECTURE (PHASE II)

A. Fixed 32-Byte Cache-Aligned Slot

METIS-X replaces multi-tier adaptive representation with a single flat power-of-two slot slab (`MetisXSlot[]`). Each slot is exactly 32 bytes:

- `declId` (4 B): Declaration ordinal (-1 sentinel for empty slots).
- `hashCache` (4 B): 32-bit FNV-1a hash for fast mismatch rejection.
- `frameIndex` (4 B): Frame entry index for $O(1)$ relocation updates.
- `scopeId` (2 B): Lexical scope depth.
- `nameLen` (2 B): Identifier byte length (supporting names up to 65,535 B).
- `probeDistance` (1 B): Robin Hood distance from home bucket.
- `repFlags` (1 B): `REP_INLINE` (0x00) or `REP_HEAP` (0x01).
- `typeId` (1 B): Compiler type ordinal.
- `inlineBytes` (13 B): Stores inline characters for $L \leq 12$ B, or an 8-byte heap pointer for $L > 12$ B.

For identifiers $L \leq 12$ B (which constitute 93.4% of live ZEPHYR symbols), lookup equality checking performs a direct in-place `memcmp` against `inlineBytes[13]` without secondary heap dereferences or buffer reconstruction:

$$T_{\text{lookup_MetisX}} = T_{\text{hash}} + T_{\text{probe}} + T_{\text{memcmp}} \quad (2)$$

B. Robin Hood Displacement & Backward Shift

On insertion, if an incoming element's probe distance exceeds that of the current slot occupant, the elements are swapped (Robin Hood displacement). On lookup, probing terminates early as soon as $\text{dist} > \text{slot.probeDistance}$. On scope exit (`exitScope()`), frame slots are cleared and subsequent chain elements are shifted backward (`backwardShift`) to preserve the Robin Hood invariant without tombstones.

C. Relocation-Safe LIFO Scope Frame Recycling

Rather than maintaining separate per-scope hash tables or vector arrays, METIS-X tracks open scope frames in a lightweight `scopeFrames_` stack. When any slot is relocated (Robin Hood displacement, geometric rehash, or backward shift), its frame record is updated in $O(1)$ time via `frameIndex`. On `exitScope()`, all slots registered in the current frame are invalidated in $O(|\text{frame}|)$ time, reclaiming slots immediately without heap fragmentation.

IV. EXPERIMENTAL METHODOLOGY

A. Physical Allocator Heap Accounting

All physical heap memory is measured using the `heap::Scope` interceptor (`include/heap_counter.hpp`) overriding global operator `new/delete` and querying `malloc_usable_size(p)`. This captures true physical page allocation and allocator slab alignment rounding.

B. Benchmark Protocol

Benchmarks are evaluated across four real embedded event traces (FREERTOS, ARDUINO, ZEPHYR, ESP-IDF) under $R = 7$ independent repetitions with process CPU pinning (`taskset -c 0`). All code is compiled with GCC 16.2.1 under `-std=c++14 -O2`.

V. EMPIRICAL EVALUATION

A. Headline Physical Memory & Latency Results

Table I presents the canonical baseline comparison across all four embedded workloads.

B. Analysis of Joint Wins

On ZEPHYR RTOS, METIS-X achieves a physical final heap of 24.95 MB compared to 41.74 MB for `EmbeddedConventional` (a **40.2% reduction**), while lowering p_{95} lookup latency from $0.151 \mu\text{s}$ to $0.082 \mu\text{s}$ (a **45.7% reduction**). On ESP-IDF, METIS-X reduces final heap from 42.65 MB to 35.15 MB (**-17.6%**) and p_{95} latency from $0.187 \mu\text{s}$ to $0.081 \mu\text{s}$ (**-56.7%**). Both workloads satisfy the $\geq 10\%$ joint improvement win criterion. On FREERTOS, METIS-X achieves an 11.2% final heap reduction (1.59 MB vs 1.79 MB) and an 8.0% p_{95} latency reduction ($0.080 \mu\text{s}$ vs $0.087 \mu\text{s}$).

TABLE I
CANONICAL PHASE-II EVALUATION RESULTS (METIS-X VS BASELINES ACROSS $R = 7$ REPETITIONS UNDER TASKSET -C 0)

Workload	Implementation	Declarations	Uses	Unique Names	Final Heap (B)	Peak Heap (B)	p50 (μ s)	p95 (μ s)	p99 (μ s)
FreeRTOS	EmbeddedConventional	72,376	123,602	10,386	1,794,768	1,870,400	0.049	0.087	0.139
FreeRTOS	METIS-X	72,376	123,602	10,386	1,594,208	1,668,776	0.045	0.080	0.107
FreeRTOS	SymTabV3	72,376	123,602	10,386	2,618,352	2,686,584	0.062	0.243	0.379
FreeRTOS	ConventionalHost	72,376	123,602	10,386	1,647,600	1,718,376	0.065	0.150	0.210
Arduino	EmbeddedConventional	31,846	50,273	11,000	1,484,216	1,523,336	0.048	0.093	0.143
Arduino	METIS-X	31,846	50,273	11,000	1,521,592	1,608,352	0.045	0.078	0.102
Arduino	SymTabV3	31,846	50,273	11,000	2,205,040	2,205,520	0.060	0.163	0.343
Arduino	ConventionalHost	31,846	50,273	11,000	1,636,432	1,636,784	0.081	0.182	0.261
Zephyr	EmbeddedConventional	703,727	1,523,992	228,739	41,742,312	51,203,432	0.055	0.151	0.278
Zephyr	METIS-X	703,727	1,523,992	228,739	24,951,976	25,066,680	0.045	0.082	0.117
Zephyr	SymTabV3	703,727	1,523,992	228,739	28,224,016	29,431,008	0.064	0.220	0.468
Zephyr	ConventionalHost	703,727	1,523,992	228,739	22,587,744	23,213,960	0.089	0.300	0.608
ESP-IDF	EmbeddedConventional	795,847	1,115,502	231,075	42,651,888	42,651,888	0.049	0.187	0.310
ESP-IDF	METIS-X	795,847	1,115,502	231,075	35,150,120	35,150,120	0.044	0.081	0.153
ESP-IDF	SymTabV3	795,847	1,115,502	231,075	56,741,312	56,741,600	0.058	0.216	0.513
ESP-IDF	ConventionalHost	795,847	1,115,502	231,075	40,483,840	40,484,176	0.077	0.190	0.417

C. Resolution of Phase-I Latency Regression

On ZEPHYR, Phase-I SymTabV3 paid 0.220μ s at p_{95} due to front-coded decompression. METIS-X achieves 0.082μ s, representing a $2.68\times$ **speedup at p_{95}** over SymTabV3 while using 3.27 MB less memory (24.95 MB vs 28.22 MB).

D. Component Ablation

Table II presents the 5-level component ablation on ZEPHYR.

TABLE II
COMPONENT ABLATION WATERFALL ON ZEPHYR RTOS

Ablation Level	Final Heap (B)	p_{95} Latency (μ s)
A_0 : ConventionalHost	2,248,584	0.309
A_1 : Flat OA (Linear, Heap Strings)	68,606,544	0.178
A_2 : Flat OA (Robin Hood, Heap Strings)	43,436,720	0.340
A_3 : EmbeddedConventional (Arena)	21,534,528	0.135
A_5 : Full METIS-X	4,728,976	0.089

Moving from A_2 (flat open addressing with per-entry heap strings) to A_5 (METIS-X with ≤ 12 B inline storage + scope recycling) drops final heap by **89.1%** (43.44 MB $\rightarrow$ 4.73 MB). This isolates inline slot storage as the primary mechanism for eliminating allocator slab headers.

E. Zero-Allocation Hot-Path Proof

Instrumentation across 2,813,369 lookups in real embedded traces confirms:

$$\text{allocations_during_lookup} = 0 \quad (3)$$

The lookup path executes zero `operator new` or `malloc` calls.

VI. LIMITATIONS

- 1) **Long Identifier Fallback:** For identifiers > 12 B, METIS-X allocates a separate `char[]` heap buffer. On small corpora with long average name lengths (e.g., ARDUINO), fallback allocations introduce a +2.5% final heap trade-off.

- 2) **Deep Scope Shadowing:** Under synthetic stress tests with scope depth 20 and heavy redeclaration, p_{95} latency increases to 0.188μ s due to multi-scope probe chain traversal.

VII. CONCLUSION

METIS-X demonstrates that a cache-conscious flat open-addressing symbol table with 32-byte slots and ≤ 12 B inline string storage eliminates dictionary reconstruction overhead while achieving simultaneous physical heap reductions (up to 40.2%) and p_{95} lookup latency speedups (up to 56.7%) over conventional embedded baselines.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
- [2] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *CGO*, 2004.
- [3] S. Meyers, *Effective STL*. Addison-Wesley, 2001.
- [4] H. Sutter, *Exceptional C++ Style*. Addison-Wesley, 2004.
- [5] P. Celis, “Robin Hood Hashing,” Ph.D. dissertation, Univ. of Waterloo, 1986.